

This section introduces RadixAttention, a novel technique for automatic and systematic KV cache reuse during runtime. Unlike existing systems that discard the KV cache after a generation request finishes, our system retains the cache for prompts and generation results in a radix tree, enabling efficient prefix search, reuse, insertion, and eviction. We implement an LRU eviction policy and a cache-aware scheduling policy to enhance the cache hit rate. RadixAttention is compatible with techniques like continuous batching [60], paged attention [23], and tensor parallelism [44]. In addition, it introduces only negligible memory and time overhead when there is no cache hit.

RadixAttention. A radix tree is a data structure that serves as a space-efficient alternative to a classical trie (prefix tree). Unlike typical trees, the edges of a radix tree can be labeled not just with single elements but also with sequences of elements of varying lengths, significantly enhancing efficiency. In our system, we utilize a radix tree to manage a mapping between sequences of tokens, and their corresponding KV cache tensors. These KV cache tensors are stored in a non-contiguous, paged layout, where the size of each page is equivalent to one token. Because GPU memory is quickly filled by the KV cache, we introduce a simple LRU eviction policy that evicts the least recently used leaf first. By evicting leaves first, we enable the re-use of their common ancestors until those ancestors become leaves and are also evicted.

这一部分解释了 RadixAttention 的核心机制：在请求结束后保留提示词和生成结果的 KV Cache，利用基数树建立 token 序列与 KV 张量之间的映射，支持共同前缀的查找、复用、插入和淘汰。基数树的边可以表示一段 token 序列，相比普通前缀树更加紧凑。KV 张量采用非连续分页存储，本文设计中每页对应一个 token。显存不足时，系统优先淘汰最近最少使用的叶节点，使共享的祖先前缀尽可能继续被复用；同时通过缓存感知调度提高缓存命中率。该机制可以与连续批处理、分页注意力和张量并行结合使用。

On MMLU, SGLang can reuse the KV cache of the 5-shot examples with RadixAttention. RadixAttention benefits both throughput and latency. RadixAttention reduces total memory usage by sharing the KV cache, allowing for a larger batch size to improve maximum throughput. RadixAttention also reduces the computation of prefill, thus decreasing the first token latency. On HellaSwag, SGLang reuses the KV cache of both few-shot examples and the common question prefix for multiple choices, resulting in two-level sharing. For the ReAct and generative agents, SGLang reuses the KV cache of the agent template and previous calls. On Tree-of-thought and Skeleton-of-thought, SGLang parallelizes the generation calls within a single program and reuses the KV cache as much as possible. On JSON decoding, SGLang accelerates decoding by decoding multiple tokens at once with a compressed finite state machine. In multi-turn chat, SGLang reuses the KV cache of the chat history. The speedup is more noticeable for short outputs because KV cache reuse mostly helps reduce the prefix time. For long outputs, because there is not much sharing between different chat sessions and the decoding time dominates, there is almost no speedup. In the DSPy RAG pipeline, SGLang reuses the KV cache of the common context example. On these benchmarks, the cache hit rate ranges from 50% to 99%. Fig. 13 (Appendix) lists the achieved and optimal cache hit rates for all of them, showing that our cache-aware scheduling approaches 96% of the optimal hit rate on average.

这一部分论文解释了 RadixAttention 通过共享 KV Cache 减少显存占用，使系统能够采用更大的批次，从而提高吞吐量；同时减少重复的 Prefill 计算，降低首 token 延迟。

Due to the decomposition in Eq. 1, the LLM can only sample and generate new tokens one by one, and the generation process of each new token depends on all the *previous tokens* in that sequence, specifically their key and value vectors. In this sequential generation process, the key and value vectors of existing tokens are often cached for generating future tokens, known as *KV cache*. Note that the KV cache of one token depends on all its previous tokens. This means that the KV cache of the same token appearing at different positions in a sequence will be different.

这部分解释了大模型会逐个生成新 token，每个新 token 的生成依赖此前的所有 token

Although fine-grained batching reduces the waste of computing and enables requests to be batched in a more flexible way, the number of requests that can be batched together is still constrained by GPU memory capacity, particularly the space allocated to store the KV cache. In other words, the serving system's throughput is *memory-bound*. Overcoming this memory-bound requires addressing the following challenges in the memory management:

Scheduling for unknown input & output lengths. The requests to an LLM service exhibit variability in their input and output lengths. This requires the memory management system to accommodate a wide range of prompt lengths. In addition, as the output length of a request grows at decoding, the memory required for its KV cache also expands and may exhaust available memory for incoming requests or ongoing generation for existing prompts. The system needs to make scheduling decisions, such as deleting or swapping out the KV cache of some requests from GPU memory.

Large KV cache. The KV Cache size grows quickly with the number of requests. As an example, for the 13B parameter OPT model [62], the KV cache of a single token demands 800 KB of space, calculated as 2 (key and value vectors) $\times$ 5120 (hidden state size) $\times$ 40 (number of layers) $\times$ 2 (bytes per FP16). Since OPT can generate sequences up to 2048 tokens, the memory required to store the KV cache of one request can be as much as 1.6 GB. Concurrent GPUs have memory capacities in the tens of GBs. Even if all available memory was allocated to KV cache, only a few tens of requests could be accommodated. Moreover, inefficient memory management can further decrease the batch size, as shown in Fig. 2. Additionally, given the current trends, the GPU's computation speed grows faster than the memory capacity [17]. For example, from NVIDIA A100 to H100, The FLOPS increases by more than 2x, but the GPU memory stays at 80GB maximum. Therefore, we believe the memory will become an increasingly significant bottleneck.

这部分论文解释了 KV Cache 占用的显存限制了请求批次大小，并会随着输出长度增加而增长，可能耗尽可用显存。因此，作者认为显存将成为越来越显著的性能瓶颈。

To address the memory challenges in §3, we introduce *PagedAttention*, an attention algorithm inspired by the classic idea of *paging* [25] in operating systems. Unlike the traditional attention algorithms, PagedAttention allows storing continuous keys and values in non-contiguous memory space. Specifically, PagedAttention partitions the KV cache of each sequence into *KV blocks*. Each block contains the key and value vectors for a fixed number of tokens,¹ which we denote as *KV*

generates a single sequence. In the remainder of this paper, we assume the more general case in which a request generates multiple sequences. In parallel sampling, one request includes multiple samples sharing the same input prompt, allowing the KV cache of the prompt to be shared as well. Via its PagedAttention and paged memory management, vLLM can realize this sharing easily and save memory.

PagedAttention 将每个序列的 KV Cache 划分为多个 KV 块。结合分页内存管理，vLLM 能让同一请求的多个采样结果共享输入提示词的 KV Cache，从而节省显存。